NAME : Leena K

ROLL NO : CH.SC.U4CSE24122

WEEK – 5

QUICK SORT:

Logic:

By first element

157 | 110 | 147 | 122 | 111 | 149 | 151 | 141 | 123 | 112 | 117 | 133

pivot element = 157

110 | 147 | 122 | 111 | 149 | 151 | 141 | 123 | 112 | 117 | 133 , 157

pivot = 110

110   147 | 122 | 111 | 149 | 151 | 141 | 123 | 112 | 117 | 133 , 157

pivot = 147

110   122 | 111 | 141 | 123 | 112 | 117 | 133   147   149 | 151   157

pivot = 122

110   111 | 112 | 117   122   123 | 133 | 141   147   149 | 151   157

pivot = 111

110 | 111 | 112 | 117 | 122 | 123 | 133 | 141 | 147 | 149 | 151 | 157

By last element

157	110	147	122	111	149	151	141	123	112	117	133

S1:

110	122	111	123	112	117	133	157	147	149	151	141

pivot=117 pivot=141

S2:

110	111	112	117	122	123	133	141	157	147	149	151

pivot=112 pivot=123 pivot=151

S3:

110	111	112	117	122	123	133	141	147	149	151	157

By random element:

S1:

157	110	147	122	111	149	151	141	123	112	117	133

pivot = 123

S2:

110	122	111	112	117	123	157	147	149	151	141	133

pivot = 151

S3:

110	111	122	112	117	123	147	149	141	133	151	157

pivot = 112 pivot = 149

S4:

110	111	112	122	117	123	147	141	133	149	151	157

pivot = 122 pivot = 149

S5:

110	111	112	117	122	123	133	141	147	149	151	157

First Element as Pivot

Code :

```c
//CH.SC.U4CSE24122
#include <stdio.h>
int partition(int arr[], int low, int high) {
    int pivot = arr[low];
    int i = low + 1;
    int j = high;
    int temp;
    while (i <= j) {
        while (i <= j && arr[i] <= pivot)
            i++;
        while (arr[j] > pivot)
            j--;
        if (i < j) {
            temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    arr[low] = arr[j];
    arr[j] = pivot;
    return j;
}
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int p = partition(arr, low, high);
        quickSort(arr, low, p - 1);
        quickSort(arr, p + 1, high);
    }
```

```c
}
int main(){
    printf("CH.SC.U4CSE24122\n");
    int arr[] = {157, 110, 147, 122, 111, 149, 151, 141, 123, 112, 117, 133};
    int n = 12;
    int i;
    quickSort(arr, 0, n - 1);
    printf("Sorted array:\n");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}
```

Output :

```
CH.SC.U4CSE24122
Sorted array:
110 111 112 117 122 123 133 141 147 149 151 157
--------------------------------
Process exited after 0.1208 seconds with return value 0
Press any key to continue . . .
```

Last Element as Pivot

Code :

```c
//CH.SC.U4CSE24122
#include <stdio.h>
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    int temp,j;
    for (j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;
    return i + 1;
}
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int p = partition(arr, low, high);
        quickSort(arr, low, p - 1);
        quickSort(arr, p + 1, high);
    }
}
int main() {
    printf("CH.SC.U4CSE24122\n");
    int arr[] = {157, 110, 147, 122, 111, 149, 151, 141, 123, 112, 117, 133};
    int n = 12;
    int i;
    quickSort(arr, 0, n - 1);
    printf("Sorted array:\n");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}
```

Output :

```
CH.SC.U4CSE24122
Sorted array:
110 111 112 117 122 123 133 141 147 149 151 157
--------------------------------
Process exited after 0.1193 seconds with return value 0
Press any key to continue . . .
```

Random Element as Pivot

Code :

```c
//CH.SC.U4CSE24122
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    int temp;
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;
    return i + 1;
}
int randomPartition(int arr[], int low, int high) {
    int r = low + rand() % (high - low + 1);
    int temp = arr[r];
    arr[r] = arr[high];
    arr[high] = temp;

    return partition(arr, low, high);
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int p = randomPartition(arr, low, high);
        quickSort(arr, low, p - 1);
        quickSort(arr, p + 1, high);
    }
}
int main() {
    printf("CH.SC.U4CSE24122\n");
    int arr[] = {157, 110, 147, 122, 111, 149, 151, 141, 123, 112, 117, 133};
    int n = 12;
    int i;
    srand(time(0));
    quickSort(arr, 0, n - 1);
    printf("Sorted array:\n");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}

```

Output :

```
CH.SC.U4CSE24122
Sorted array:
110 111 112 117 122 123 133 141 147 149 151 157
--------------------------------
Process exited after 0.1714 seconds with return value 0
Press any key to continue . . .
```

Conclusion:

Based on the array order, position of the pivot element changes.

So, the efficiency doesn't depend on the position of the pivot element changes